Algoritmike dhe programim i avancuar

Leksioni 3

Ideja e Klasave (Java, C++)

- Me një klasë, specifikojmë një skemë për një ose më shumë objekte. Për shembull, në Java (shih librin faqe 16):

```
class thermostat
{
private float currentTemp();
private float desiredTemp();

public void furnace_on()
{
// trupi i metodës vjen këtu
}

public void furnace_off()
{
// trupi i metodës vjen këtu
}
} // fundi i klasës thermostat
```

Instancat e një Klase

- Specifikimi I nje klase nuk krijon nje objekt ne ate klase.
- Java I krijon objektet duke perdorur **new,** dhe ruan nje reference te kesaj instance ne nje variabel

```
thermostat therm1, therm2; // krijo dy referenca
therm1 = new thermostat(); // krijo dy objekte dhe
therm2 = new thermostat(); // ruaj referencat për to
```

Thirrja e Metodave të një Objekti

- Pjesë të tjera të programit ndërveprojnë me metodat e një objekti, jo me të dhënat e tij.
- Për t'i thënë objektit therm2 të ndezë ngrohjen, do të shkruanim:

```
therm2.furnace_on();
```

- Operatori pikë (.) lidh një objekt me një nga metodat e tij.

Përmbledhje

1. Objektet përmbajnë si metoda ashtu edhe fusha (të dhëna).
2. Një klasë është një specifikim për çdo numër objektesh.
3. Për të krijuar një objekt, përdorni fjalën kyçe new së bashku me emrin e klasës.
4. Për të thirrur një metodë për një objekt të caktuar, përdorni operatorin pikë.

Trashëgimia

- Krijimi i një klase, e quajtur klasa bazë
- Krijimi i një klase tjetër, e quajtur klasa e derivuar
  - Ka të gjitha karakteristikat e bazës, plus disa karakteristika shtesë.
- Shembull:
  - Një klasë bazë **Animal** mund të ketë metoda të lidhura si eat() dhe run() si dhe variablën name, ndërsa një klasë e prejardhur **Dog** mund të trashëgojë nga **Animal**, duke fituar këto metoda dhe duke shtuar një metodë të re swim().

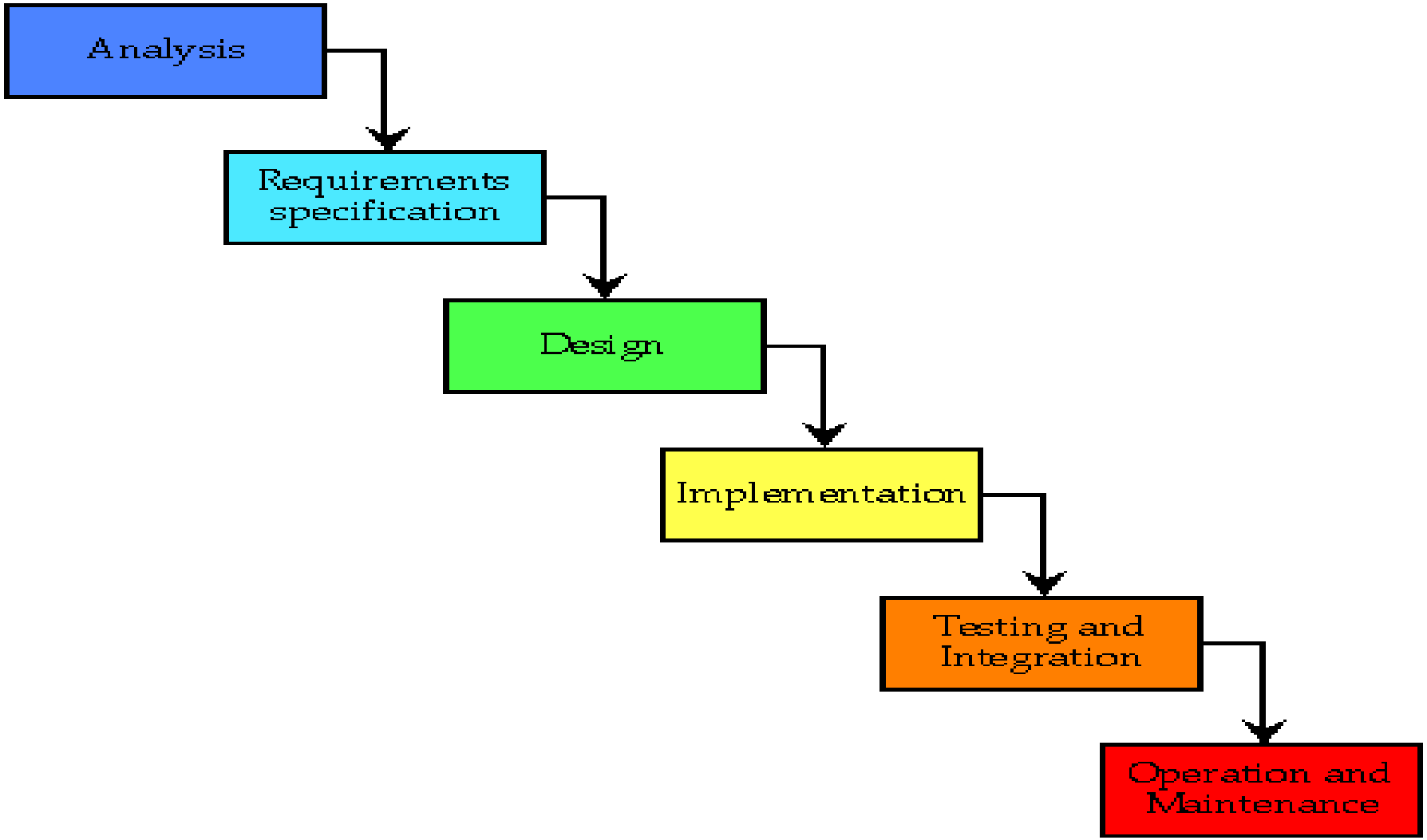
Trashëgimia ju mundëson të shtoni lehtësisht karakteristika në një klasë ekzistuese dhe është një ndihmë e rëndësishme në dizajnimin e programeve me shumë klasa të lidhura.

Inxhinieria e Softuerit

- Inxhinieria e softuerit është studimi i mënyrave për të krijuar programe kompjuterike të mëdha dhe komplekse, që përfshijnë shumë programues.

- Fokusohet në dizajnin e përgjithshëm të programeve dhe në krijimin e atij dizajni nga nevojat e përdoruesve fundorë.

Modeli Ujëvarë

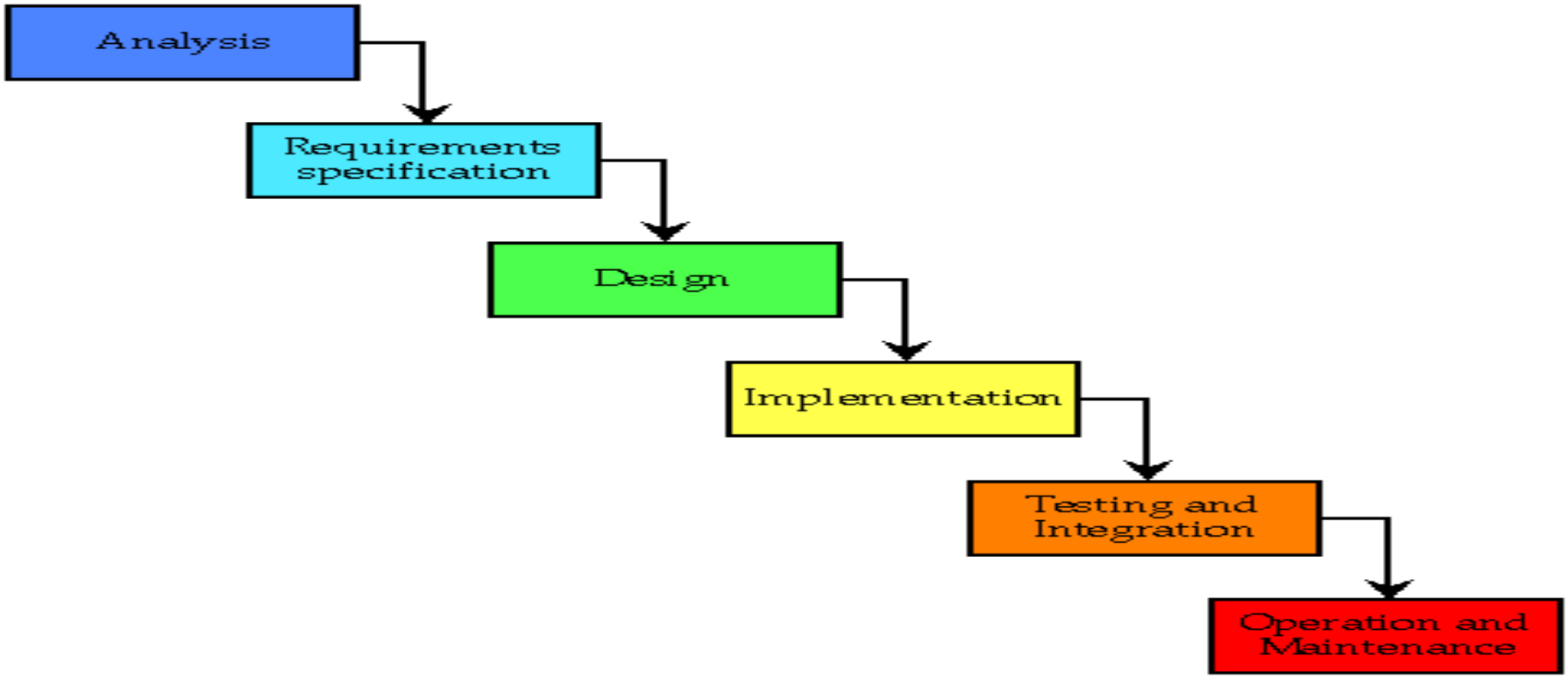


Inxhinieria e Softuerit

- Cikli i jetës së një projekti softueri përbëhet nga sa vijon:
  1. Specifikimi – qëllimi i softuerit, kërkesat, etj.
  2. Dizajni – komponentët dhe ndërveprimi i softuerit
  3. Verifikimi – komponentët individualë dhe funksionaliteti global
  4. Kodimi – shkrimi aktual i softuerit
  5. Testimi – validimi i funksionalitetit të duhur
  6. Prodhimi – shpërndarja te komuniteti
  7. Mirëmbajtja – përditësimi i softuerit (FYI në një dizajn të dobët, këto kosto mund të jenë të larta!!)

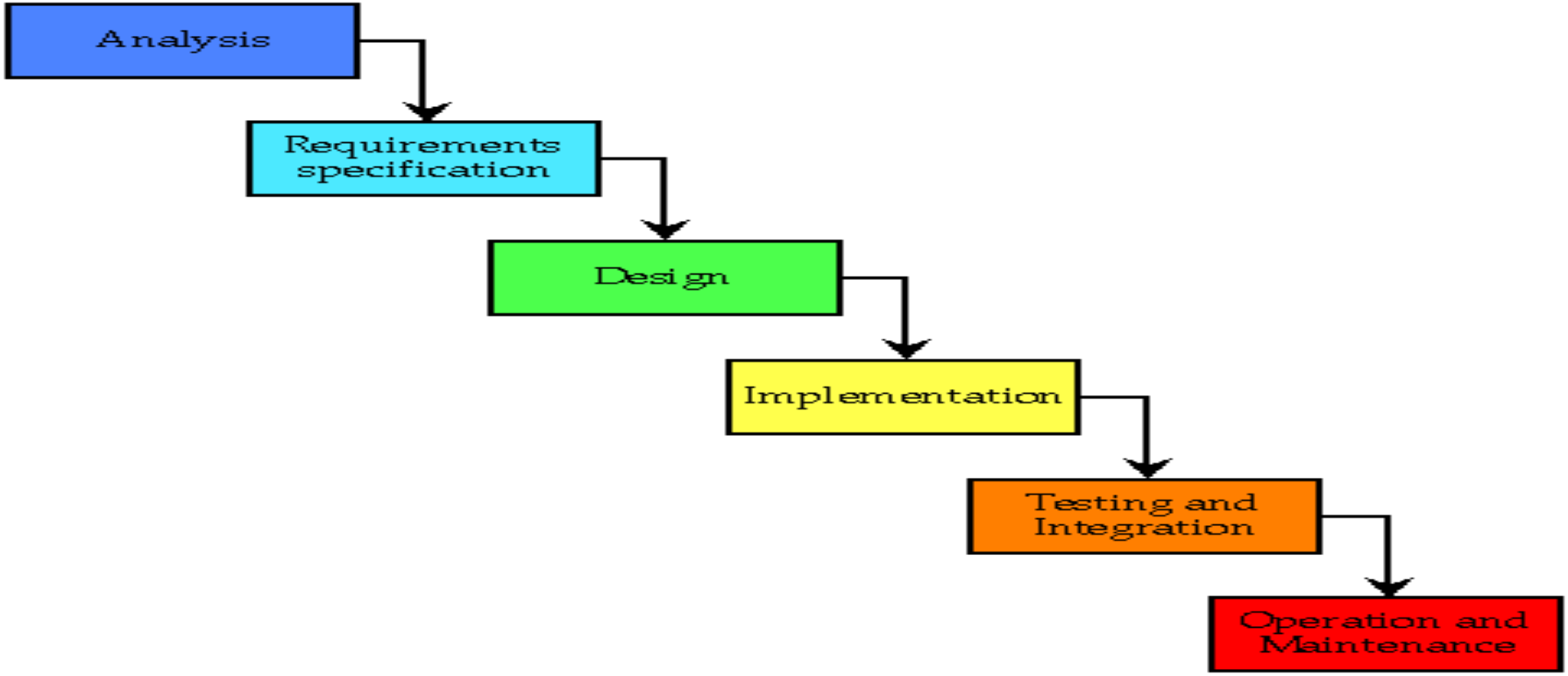
Faza 1: Specifikimi

- Këtu i përgjigjemi pyetjeve se çfarë qëllimi shërben softueri, dhe kërkesat. Nivel shumë i lartë në këtë fazë.



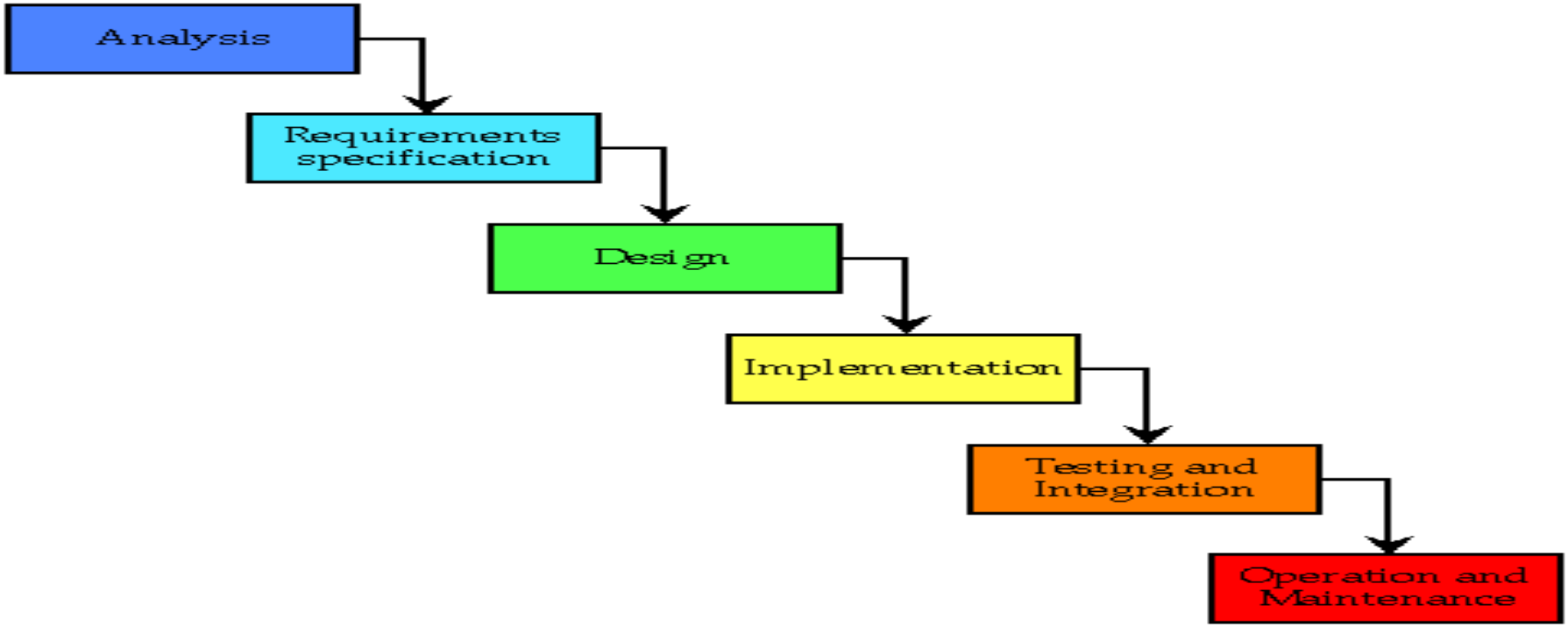
Faza 2: Dizajni

- Në këtë fazë, ndajmë softuerin në komponentë dhe përshkruajmë ndërveprimin e tyre.



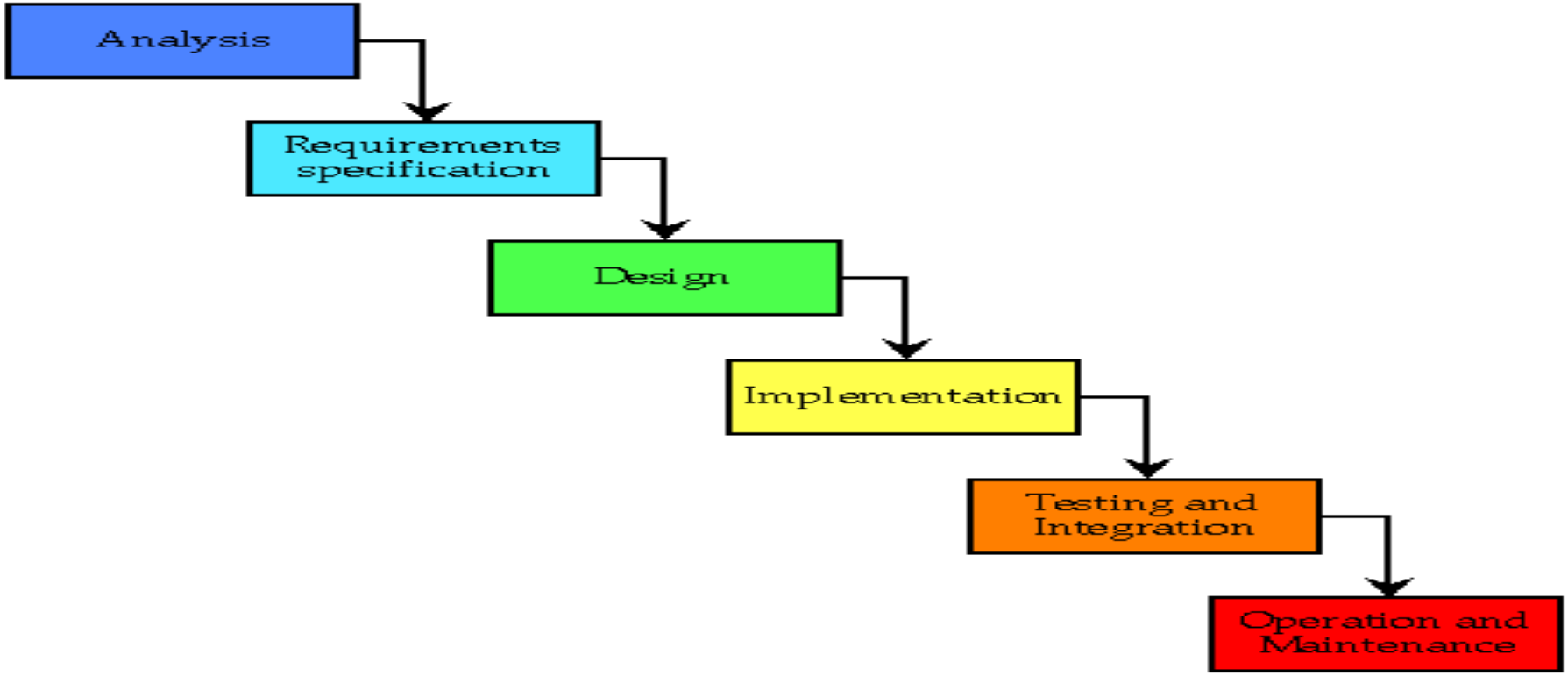
Faza 3: Verifikimi

- Verifikimi përfshin një rishikim të të gjithë komponentëve, studimin e hyrjeve dhe daljeve të tyre individuale dhe sigurimin e funksionalitetit të dizajnit.



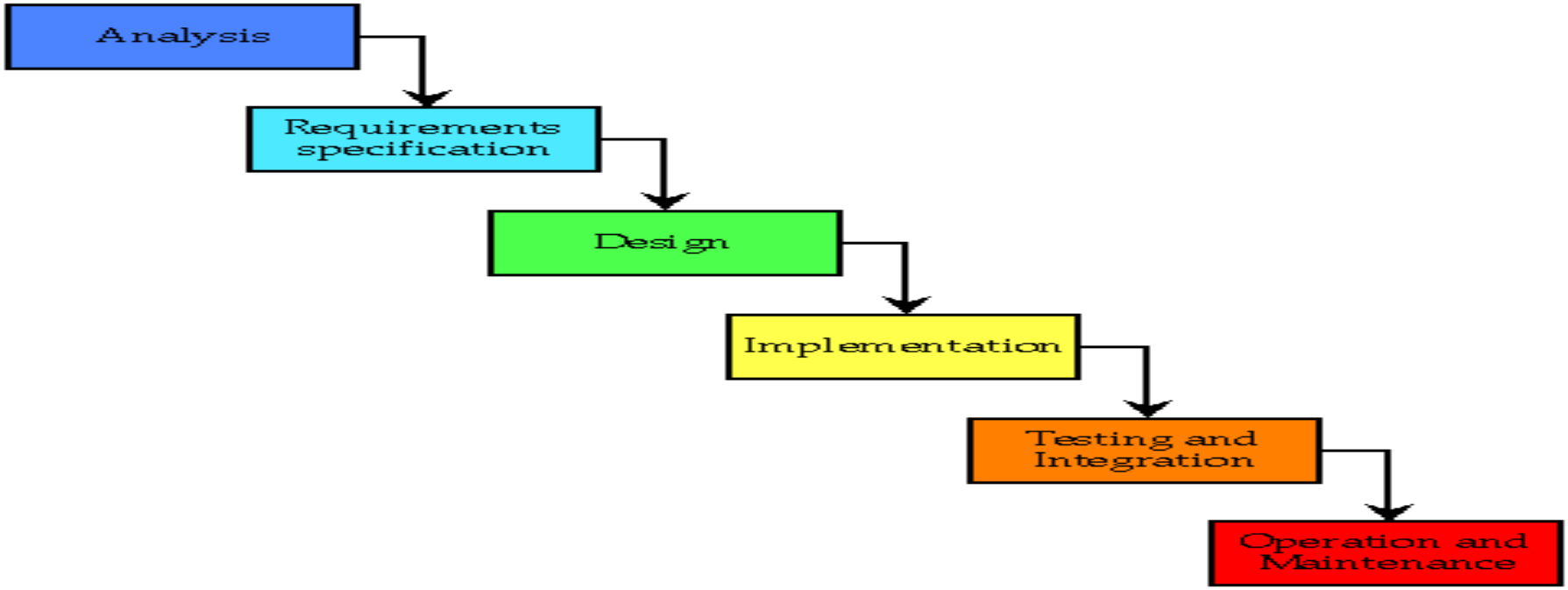
Faza 4: Kodimi

- Përfshin shkrimin aktual të softuerit.



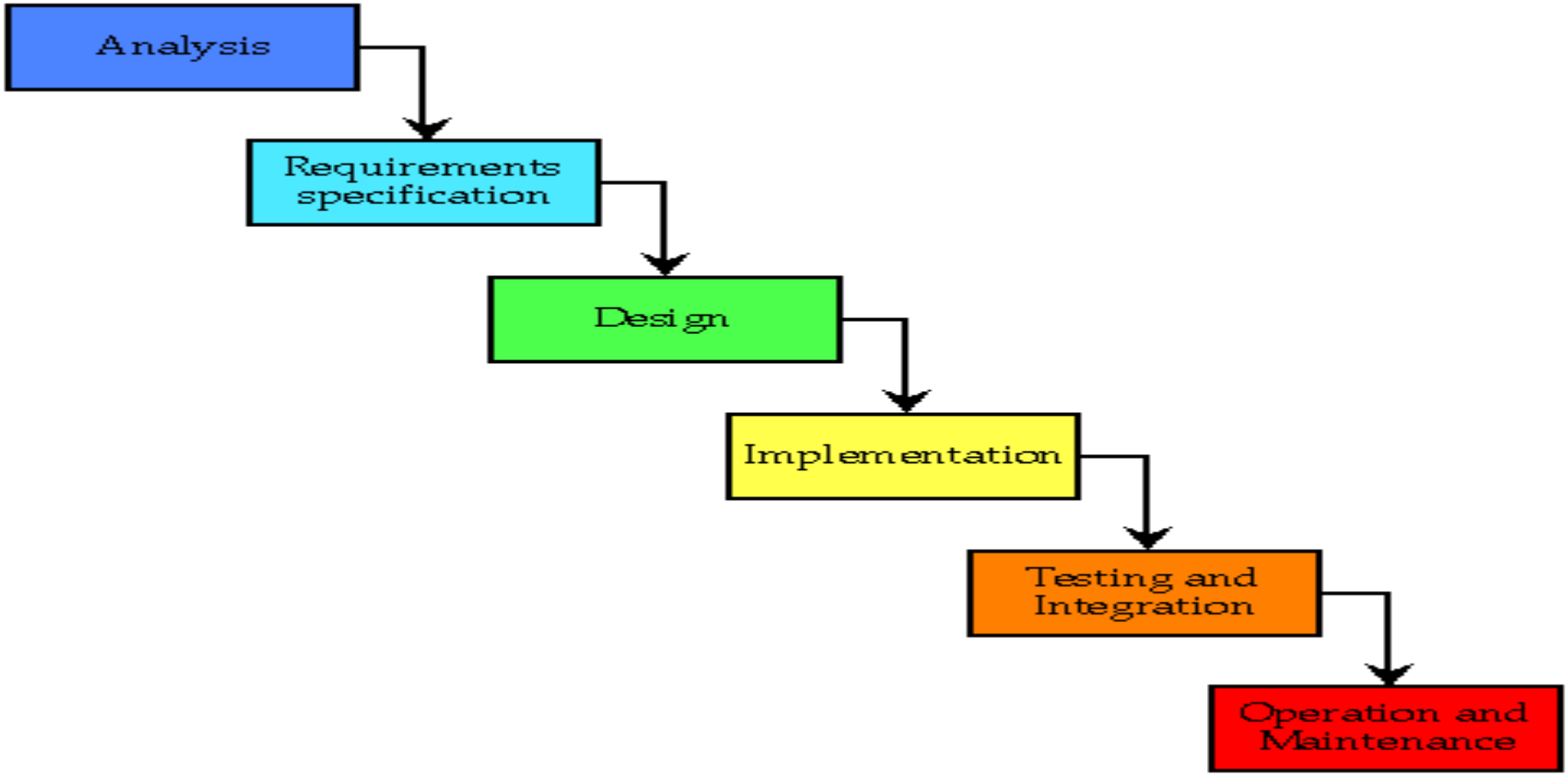
Faza 5: Testimi

- Involves running the software package through a set of benchmarks, which make up a testsuite, and verify proper functionality. You can also test individual components.



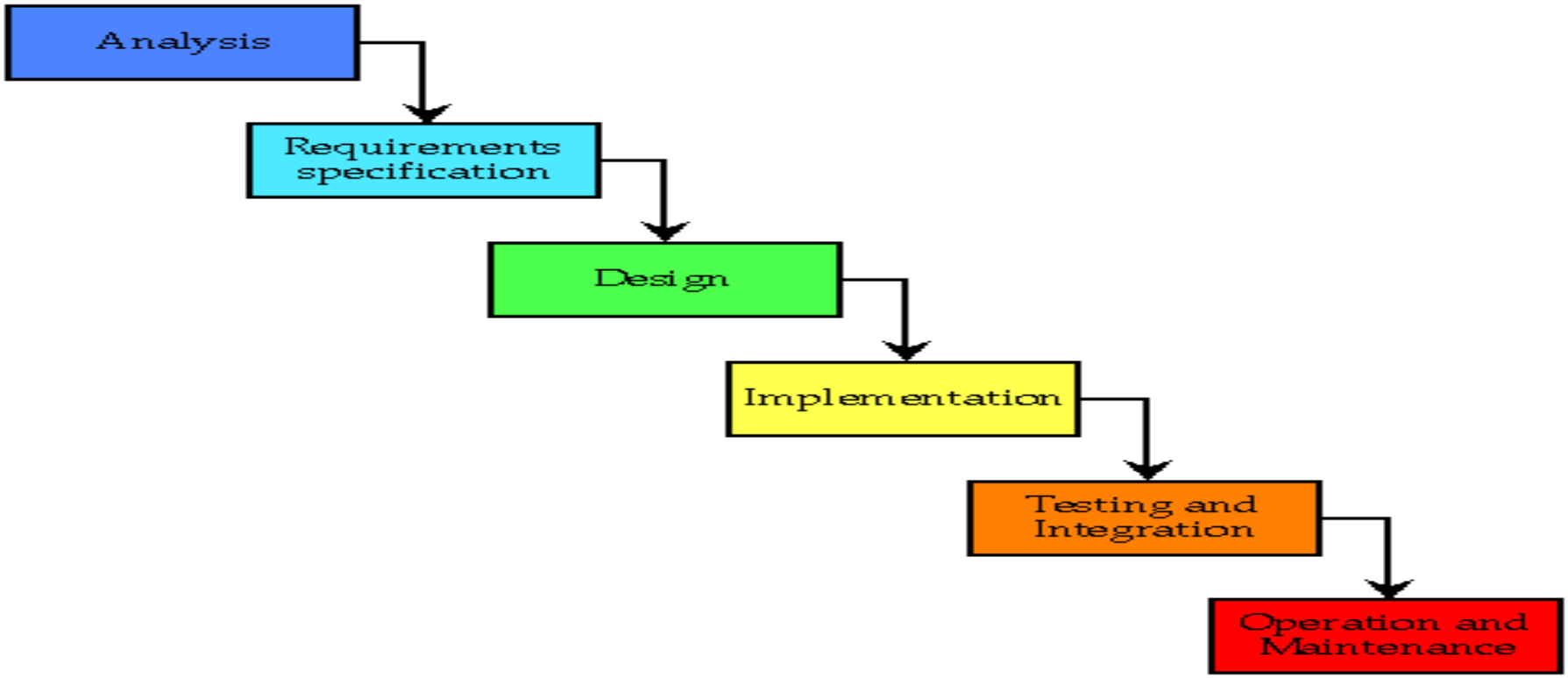
Faza 6: Prodhimi

- Shpërndarja e softuerit te komuniteti.



Faza 7: Mirëmbajtja

- Përditësimi i softuerit duke mbajtur dizajnin e brendshëm të paprekur.



Rishikimi Final i disa Koncepteve të Java-s

- Dallimi midis një vlere dhe një reference:

int intVar;
BankAccount bc1;

- Cila është vlerë dhe cila është referencë?
- Si e kuptoi interpretuesi t'i alokojë ato ndryshe?
- Cfare duhet te bej perpara se te perdor **bc1**?

Madhësitë Primitive dhe Intervalet e Vlerave

Data Type	Default Value (for fields)	Size (in bits)	Minimum Range	Maximum Range
byte	0	Occupy 8 bits in memory	-128	+127
short	0	Occupy 16 bits in memory	-32768	+32767
int	0	Occupy 32 bits in memory	-2147483648	+2147483647
long	0L	Occupy 64 bits in memory	-9223372036854775808	+9223372036854775807
float	0.0f	Occupy 32-bit IEEE 754 floating point	1.40129846432481707e-45	3.40282346638528860e+38
double	0.0d	Occupy 64-bit IEEE 754 floating point	4.94065645841246544e-324d	1.79769313486231570e+308d
char	'\u0000'	Occupy 16-bit, unsigned Unicode character		0 to 65,535
boolean	false	Occupy 1- bit in memory	NA	NA

Burimi: [illegible]india.net

Shembull (fq. 18)

- class BankApp
- {
- public static void main(String[] args)
- {
- BankAccount ba1 = new BankAccount(100.00); // krijo llogarinë
- System.out.print("Before transactions, ");
- ba1.display(); // shfaq balancën
- ba1.deposit(74.35); // bëj depozitë
- ba1.withdraw(20.00); // bëj tërheqje
- System.out.print("After transactions, ");
- ba1.display(); // shfaq balancën
- } // fundi main()
- } // end class BankApp

```
class BankAccount
{
private double balance; // balanca e llogarisë
public BankAccount(double openingBalance) // konstruktori
{
balance = openingBalance;
}
public void deposit(double amount) // bën depozitë
{
balance = balance + amount;
}
public void withdraw(double amount) // bën tërheqje
{
balance = balance – amount;
}
public void display() // shfaq balancën
{
System.out.println("balance=" + balance);
}
} // end class BankAccount
```

Shembulli i bankës

- Ka dy klasa në bank.java.
- The first one, BankAccount, contains the fields and methods for our bank account.
- The second class, BankApp, plays a special role.
- The main() method creates an object of class BankAccount, initialized to a value of
- 100.00, që është balanca hapëse, me këtë deklaratë:

```
BankAccount ba1 = new BankAccount(100.00);
                    // krijo llogarinë
```

Shembulli i bankës

- The System.out.print() method displays the string used as its argument, Before
- transactions:, dhe llogaria shfaq balancën e saj me këtë deklaratë:

```
ba1.display();
```

- Programi pastaj bën një depozitë në, dhe një tërheqje nga, llogaria:

```
ba1.deposit(74.35);
ba1.withdraw(20.00);
```

- Së fundmi, programi shfaq balancën e re të llogarisë dhe përfundon.

Konstruktori

- Klasa **BankAccount** përfshin gjithashtu një **konstruktor**, i cili është një metodë e veçantë që thirret automatikisht sa herë që krijohet një objekt i ri.
- Një konstruktor ka gjithmonë saktësisht të njëjtin emër si klasa, prandaj ky quhet BankAccount().
- Ky konstruktor ka një argument, i cili përdoret për të vendosur balancën hapëse kur krijohet llogaria.
- Një konstruktor lejon që një objekt i ri të inicializohet në një mënyrë të përshtatshme